Convolutional Neural Networks and Vision Transformers for Natural and Urban Scene Classification

Jauregui Arvilla Josue Emmanuel
Universidad Panamericana Campus Aguascalientes
Aguascalientes, Mexico
josuew2013@gmail.com

***Abstract*—This paper presents a comparative study of deep learning architectures for scene classification using the Intel Image Classification dataset. We evaluate three distinct paradigms: ResNet-50, EfficientNet-B0, and Vision Transformer (ViT-B/16). To address inherent dataset challenges, we implement a robust preprocessing pipeline with data augmentation, ImageNet normalization, and a class-weighting strategy to mitigate class imbalance. Furthermore, we incorporate Grad-CAM interpretability analysis to inspect spatial decision-making. Our experimental results outline the trade-offs between accuracy, computational efficiency, and memory footprint, providing insights for edge and cloud deployment.**

***Index Terms*—image classification, CNN, computer vision, Intel Image Classification, Pytorch**

I. Introduction

The automatic classification of natural and urban scenes represents a cornerstone in modern computer vision. With critical applications in sectors such as autonomous vehicle navigation, environmental monitoring, service robotics, and smart city development, the ability of an artificial system to accurately categorize complex environments—such as mountainous landscapes, glaciers, dense forest zones, or urban streets—has become an indispensable requirement.

Historically, this task has been dominated by Convolutional Neural Networks (CNNs), which exploit a strong spatial inductive bias through local convolution operations. Architectures like ResNet-50 have established themselves as robust baseline standards thanks to their residual connections (skip connections), which effectively mitigate the vanishing gradient problem in deep networks. Subsequently, the evolution toward optimized architectures like EfficientNet demonstrated that it is possible to maximize representational efficiency through the compound scaling of depth, width, and resolution. However, in recent years, Vision Transformers (ViTs) have introduced a radical paradigm shift by replacing convolutional filters with image patches and Global Self-Attention mechanisms, allowing models to capture long-range spatial dependencies right from the earliest network layers.

Despite these remarkable theoretical advances, training vision models on real-world datasets introduces significant practical challenges. These include class distribution imbalances, high variability in lighting conditions, and the presence of background noise irrelevant to the classification task. To counter these limitations, it is essential to implement rigorous preprocessing pipelines, including controlled data augmentation techniques, standardized normalization, and adaptive weighting strategies in loss functions (such as weighted CrossEntropyLoss), ensuring that the model does not develop biases toward majority classes.

In this context, this paper provides a comparative study that empirically evaluates the performance, computational efficiency, and interpretability of three remarkably distinct architectural paradigms: ResNet-50, EfficientNet-B0, and ViT-B/16, using the Intel Image Classification dataset as a benchmark. Through a unified experimental protocol, we analyze the critical trade-offs between classification accuracy, VRAM memory consumption, convergence time, and generalization capacity. Furthermore, we complement the quantitative evaluation with an interpretability analysis using Grad-CAM activation maps, aiming to verify whether the models learn semantically relevant visual patterns or if they rely on spurious spatial artifacts.

II. Methods

A. Dataset

The Intel Image Classification Dataset was as our benchmark because it provides a diverse and balanced mixture of natural landscapes and man-made urban environments. This dual nature challenges models to capture both fine local textures and global structural context. Furthermore, its manageable scale enables efficient experimentation across different architectural paradigms (CNNs and Vision Transformers) while presenting realistic computer vision challenges, such as subtle class ambiguities and distribution variances.

The dataset contains 24,335 RGB images belonging to six landscape categories:

- Buildings
- Forest
- Glacier
- Mountain

- Sea
- Street

B. Preprocessing

1) **Image Resizing** All images were resized to 224×224 pixels to ensure compatibility with the fixed spatial input size expected by the pretrained architectures.
2) **Random Horizontal Flipping** Flips the image horizontally with a 50% probability ($p = 0.5$).
3) **Random Rotation**: Randomly rotates the image by up to $\pm 15^\circ$ to account for natural variations in camera angles.
4) **Color Jitter**: Randomly adjusts brightness and contrast by a factor of 0.1 to simulate diverse illumination conditions.
5) **Tensor Conversion and Normalization**: Images are converted to PyTorch Tensors and normalized using standard ImageNet statistics (mean $= [0.485, 0.456, 0.406]$, std $= [0.229, 0.224, 0.225]$), which stabilizes gradients and accelerates convergence.

C. Dataset Splitting

The Intel Image Classification dataset is partitioned into predefined training and validation subsets. As summarized in Table I, the training set comprises 14,034 images (82.39%), while the validation set consists of 3,000 images (17.61%), making a total of 17,034 samples across all six distinct classes: buildings, forest, glacier, mountain, sea, and street. This allocation ensures that models have a sufficiently diverse and robust volume of data for feature learning during training, while providing an independent and reliable set for unbiased performance evaluation.

TABLE I
DATA DISTRIBUTION

Dataset	Number of Images	Percentage
Training	14034	82.39%
Validation	3000	17.61%
Total	**17034**	**100%**

D. ResNet-50 Architecture

To address the degradation problem in deep architectures—where deeper networks exhibit higher training error—the ResNet-50 architecture [2], introduced by Microsoft Research, incorporates a deep residual learning framework. Rather than forcing stacked layers to directly fit an underlying mapping $\mathcal{H}(x)$, the network parameterizes them to learn a residual mapping $\mathcal{F}(x) = \mathcal{H}(x) - x$, reformulated via identity shortcut connections as $\mathcal{H}(x) = \mathcal{F}(x) + x$. Unlike lighter variants, ResNet-50 utilizes a bottleneck block design comprising 1×1, 3×3, and 1×1 convolutions to reduce computational complexity. The network is structured into five convolutional stages, followed by a Global Average Pooling (GAP) layer and a final fully connected classification layer.

E. EfficientNet-B0 Architectural Analysis

EfficientNet-B0, developed by Google Research, was selected due to its optimal balance between parameter efficiency and feature extraction capability, achieved through compound scaling [1]. Unlike conventional convolutional networks that scale depth, width, or resolution independently, EfficientNet-B0 uniformly scales all three dimensions using a fixed compound coefficient. This architectural choice prevents early saturation of feature representation while minimizing computational complexity and floating-point operations (FLOPs).

The core structural building block of the network is the Mobile Inverted Bottleneck Convolution (MBConv). Each MBConv block utilizes depthwise separable convolutions, which decouple spatial filtering from channel-wise feature combination. This factorization substantially reduces the overall parameter count compared to standard convolutions. Furthermore, these blocks incorporate Squeeze-and-Excitation (SE) modules that perform dynamic channel-wise recalibration, allowing the network to selectively emphasize highly informative feature maps while suppressing redundant visual noise.

Non-linearity across the feature extraction pipeline is introduced via the Swish activation function, defined as $f(x) = x \cdot \sigma(x)$. The smooth, non-monotonic gradient behavior of Swish accelerates training convergence and improves gradient propagation across deeper layers. Through these architectural mechanisms, EfficientNet-B0 achieves high representation capacity with only 5.3 million parameters, providing a lightweight yet powerful feature extractor for image classification.

F. Vision Transformer (ViT-B/16) Architectural Analysis

The Vision Transformer (ViT-B/16), developed by Google Research, represents a paradigm shift in computer vision by adapting the standard Transformer architecture—originally designed for natural language processing—directly to image classification [3]. Unlike traditional Convolutional Neural Networks (CNNs) that rely on local receptive fields and hardcoded inductive biases such as translation equivariance, ViT-B/16 models global context across the entire image right from the initial layers using Multi-Head Self-Attention (MHSA) mechanisms.

The architecture processes input images by partitioning them into a sequence of non-overlapping 16×16 pixel spatial patches. Each patch is flattened and linearly projected into a D-dimensional vector space ($D = 768$). Because the self-attention operation is permutation-invariant, learnable 1D positional embeddings are added to the patch projections to preserve spatial geometry. Furthermore, a learnable classification token (`[CLS]`) is prepended to the sequence; its state at the final layer serves as the consolidated global image representation fed into the classification head.

The core backbone consists of 12 Transformer encoder layers, each incorporating Multi-Head Self-Attention, Layer Normalization (LayerNorm), and a Multi-Layer Perceptron (MLP) block equipped with Gaussian Error Linear Unit (GELU) activations, structured with residual skip connections. With approximately 86 million parameters, ViT-B/16 trades off local

translational biases for large-scale representational capacity, effectively capturing long-range semantic dependencies and contextual relationships across the entire visual field.

III. Experimental Results and Analysis

A. Hardware and Software Environment

All training and evaluation experiments were conducted on a workstation equipped with an AMD Ryzen 9 7950X3D processor, 64 GB of RAM at 6000 MHz, and an NVIDIA GeForce RTX 4090 GPU. The framework used was PyTorch under CUDA 12.1.

B. Training Hyperparameters

The hyperparameters used during model training are summarized in Table II.

TABLE II
Training Hyperparameters

Hyperparameter	Value
Optimizer	AdamW
Learning Rate	1×10^{-4}
Batch Size	32
Epochs	10
Loss Function	Weighted Cross-Entropy Loss
Initial Weights	ImageNet pretrained
Framework	PyTorch

C. Performance Evaluation Metrics

The evaluation metrics and efficiency indicators are defined as follows:

- **Accuracy:** Measures the overall proportion of correctly classified images across all categories.
- **F1-Score:** Represents the harmonic mean of precision and recall calculated independently for each class and averaged uniformly, providing a robust measure against class imbalance.
- **Parameters (Params):** Quantifies the total number of learnable weights in the architecture, indicating model complexity.
- **Training Time (s):** Measures the total time in seconds required to complete the model training process across all epochs.
- **Peak VRAM Usage (MB):** Captures the maximum GPU memory allocated during training, measured using `torch.cuda.max_memory_allocated`.

D. Quantitative Performance Comparison

Table III summarizes the quantitative performance metrics, architecture parameters, total training time, and peak GPU VRAM consumption.

TABLE III
Performance and Resource Efficiency Comparison Between Architectures

Model	Train Acc.	Val. Acc.	F1-Score	Parameters	Time (s)	VRAM (MB)
ResNet-50	0.9862	0.9387	0.9396	23,520,326	1438.88	2958.55
EfficientNet-B0	0.9848	0.9333	0.9345	4,015,234	1372.45	2973.57
ViT-B/16	0.9691	0.9300	0.9312	85,803,270	1809.19	5165.87

E. Discussion and Model Trade-offs

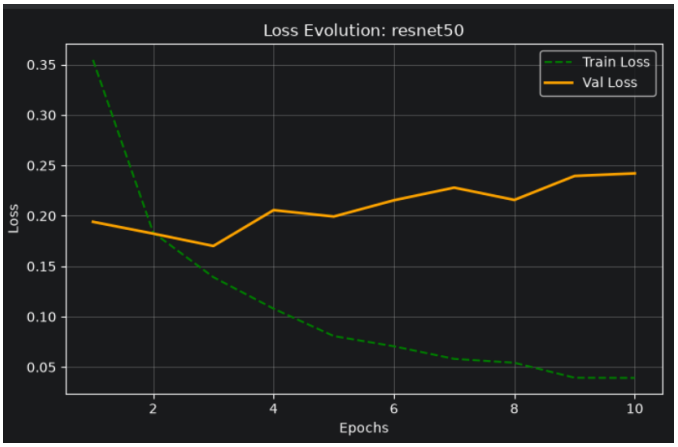
- **ResNet-50:** Achieved the highest validation accuracy (93.87%) and top F1-Score (0.9396), demonstrating a robust performance balance with 23.5M parameters and a training time of $1,438.88$ s.
- **EfficientNet-B0:** Proved to be the most parameter-efficient model (4.01M parameters), being approximately $6\times$ lighter than ResNet-50 and $21\times$ lighter than ViT-B/16. It reached competitive validation accuracy (93.33%) with the fastest overall training time ($1,372.45$ s).
- **Vision Transformer (ViT-B/16):** Required the largest computational footprint (85.8M parameters and $5,165.87$ MB of VRAM) and the longest training time ($1,809.19$ s), while obtaining the lowest validation accuracy (93.00%) among the tested architectures.

F. Discussion and Model Trade-offs

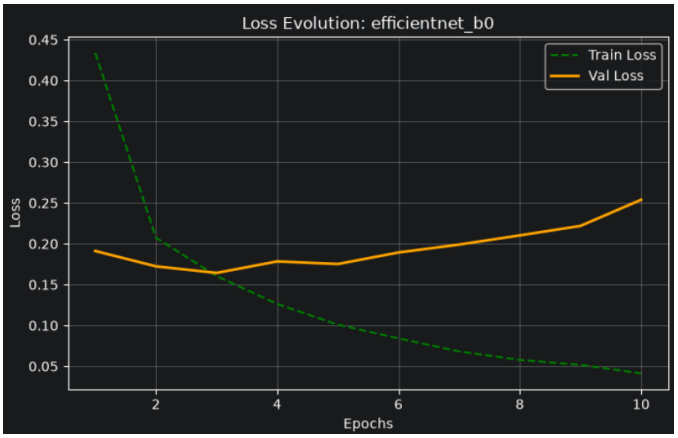
- **ResNet-50:** Achieved the highest validation accuracy (93.87%) and top F1-Score (0.9396), demonstrating a robust performance balance with 23.5M parameters and a training time of $1,438.88$ s.
- **EfficientNet-B0:** Proved to be the most parameter-efficient model (4.01M parameters), being approximately $6\times$ lighter than ResNet-50 and $21\times$ lighter than ViT-B/16. It reached competitive validation accuracy (93.33%) with the fastest overall training time ($1,372.45$ s).
- **Vision Transformer (ViT-B/16):** Required the largest computational footprint (85.8M parameters and $5,165.87$ MB of VRAM) and the longest training time ($1,809.19$ s), while obtaining the lowest validation accuracy (93.00%) among the tested architectures.

G. Loss Evolution and Convergence Analysis

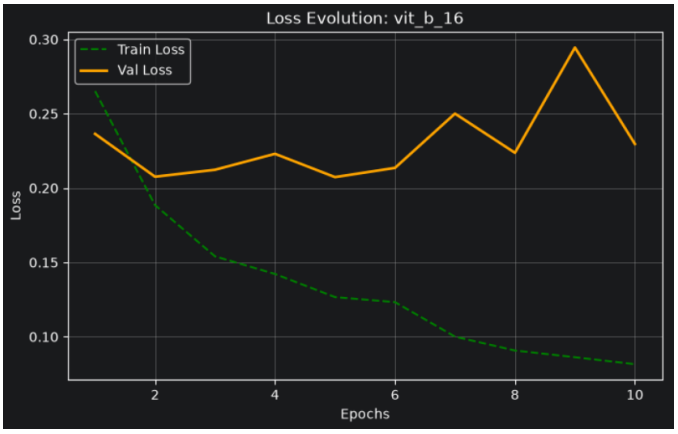
Both ResNet-50 and EfficientNet-B0 reached optimal validation loss around epoch 3 ($\approx$ 0.1701 and $\approx$ 0.1643, respectively). Beyond epoch 4, continuous training led to mild overfitting, as evidenced by decreasing training loss alongside rising validation loss. ViT-B/16 exhibited higher validation loss instability throughout training.



(a) ResNet-50 loss evolution



(b) EfficientNet-B0 loss evolution



(c) ViT-B/16 loss evolution

Fig. 1. Training dynamics over 10 epochs for the evaluated models.

H. Experimental Results and Model Comparison

To evaluate and compare ResNet-50, EfficientNet-B0, and ViT-B/16, two metrics were analyzed across 10 training epochs: **Validation Loss** (error rate on new images) and **Validation Accuracy** (percentage of correct classifications).

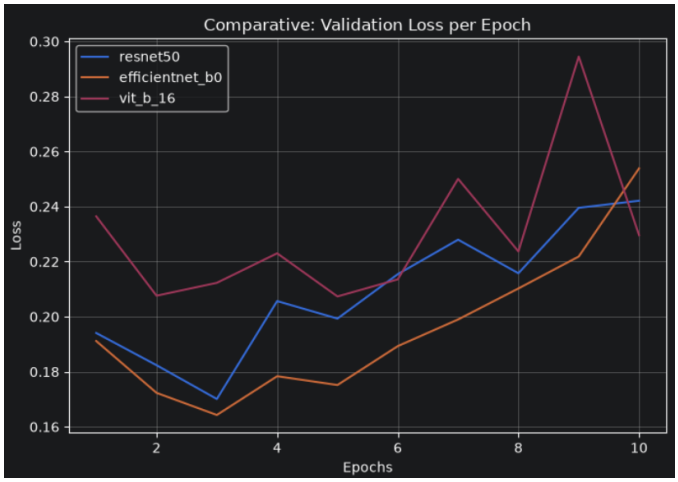


Fig. 2. Validation Loss per epoch for ResNet-50, EfficientNet-B0, and ViT-B/16.

1) Analysis of Validation Loss (Error Rate): Validation loss measures how many mistakes the model makes on images it has never seen before. A lower value indicates a better model.

As shown in Fig. 2:

- **ResNet-50 and EfficientNet-B0:** Both models perform similarly. They reach their lowest error rate at epoch 3 ($\approx$ 0.170 for ResNet-50 and $\approx$ 0.164 for EfficientNet-B0). After epoch 3, the error slowly increases because the models begin to memorize training details rather than general patterns.
- **ViT-B/16:** Exhibits unstable behavior throughout training. Its error curve fluctuates significantly, reaching a high peak of 0.295 at epoch 9, making it less reliable.

2) Analysis of Validation Accuracy (Percentage of Success): Validation accuracy shows the percentage of test images correctly identified by each model. A higher value indicates better classification performance.

As depicted in Fig. 3:

- **ResNet-50:** Reaches the highest success rate overall, achieving a peak accuracy of $\approx 94.1\%$ at epochs 5 and 8.
- **EfficientNet-B0:** Demonstrates the most consistent performance. It rapidly reaches $\approx 93.9\%$ accuracy at epoch 3 and remains stable above 93.3% across all subsequent epochs.
- **ViT-B/16:** Achieves the lowest performance among the three models, oscillating between 91.5% and 93.0%, with sharp drops such as falling to 91.2% at epoch 9.

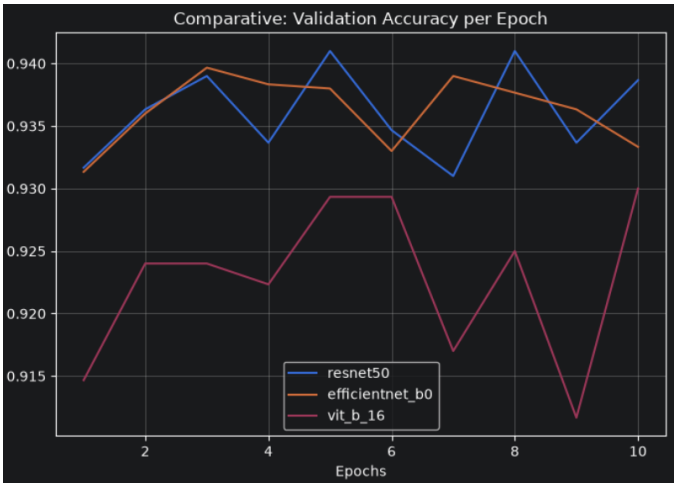


Fig. 3. Validation Accuracy per epoch for ResNet-50, EfficientNet-B0, and ViT-B/16.

3) Main Conclusions:

- **When to stop training:** Training should be stopped at **epoch 3**. Continuing beyond epoch 3 increases error without providing significant accuracy gains.
- **Best overall model: EfficientNet-B0** is the most practical choice because it is lightweight, fast, and achieves accuracy nearly identical to ResNet-50.

I. Visual Explanation with Grad-CAM

To understand where the model looks when making a classification, Gradient-weighted Class Activation Mapping (Grad-CAM) was applied to the **ResNet-50** model. Grad-CAM generates a heatmap over the input image: **red areas** highlight the regions most important for the model's decision, while **blue areas** are ignored.

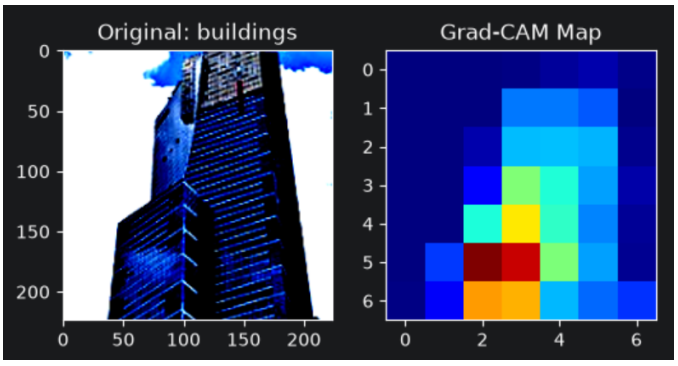


Fig. 4. Grad-CAM visualization for ResNet-50: (left) Original input image and (right) generated heatmap.

1) ResNet-50 Focus Analysis: As shown in Fig. 4, the heatmap confirms that ResNet-50 focuses directly on the main object in the image while ignoring the background. This shows that the model bases its decisions on relevant object features rather than background noise.

Due to differences in architecture and attention mechanisms, extending this visual analysis to EfficientNet-B0 and ViT-B/16 is left as future work.

REFERENCES

[1] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6105–6114.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[3] A. Dosovitskiy *et al.*, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.